TEL354 - Minería de datos

Clase 4: Exploratory visualization

Prof. Nicolás Torres
nicolas.torresr@usm.cl
Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

In [ ]:
```
from Clase4 import *
```

In [2]:
```
titanic_df = pd.read_csv('data/titanic.csv')
```

In [3]:
```
titanic_df
```

Out[3]:

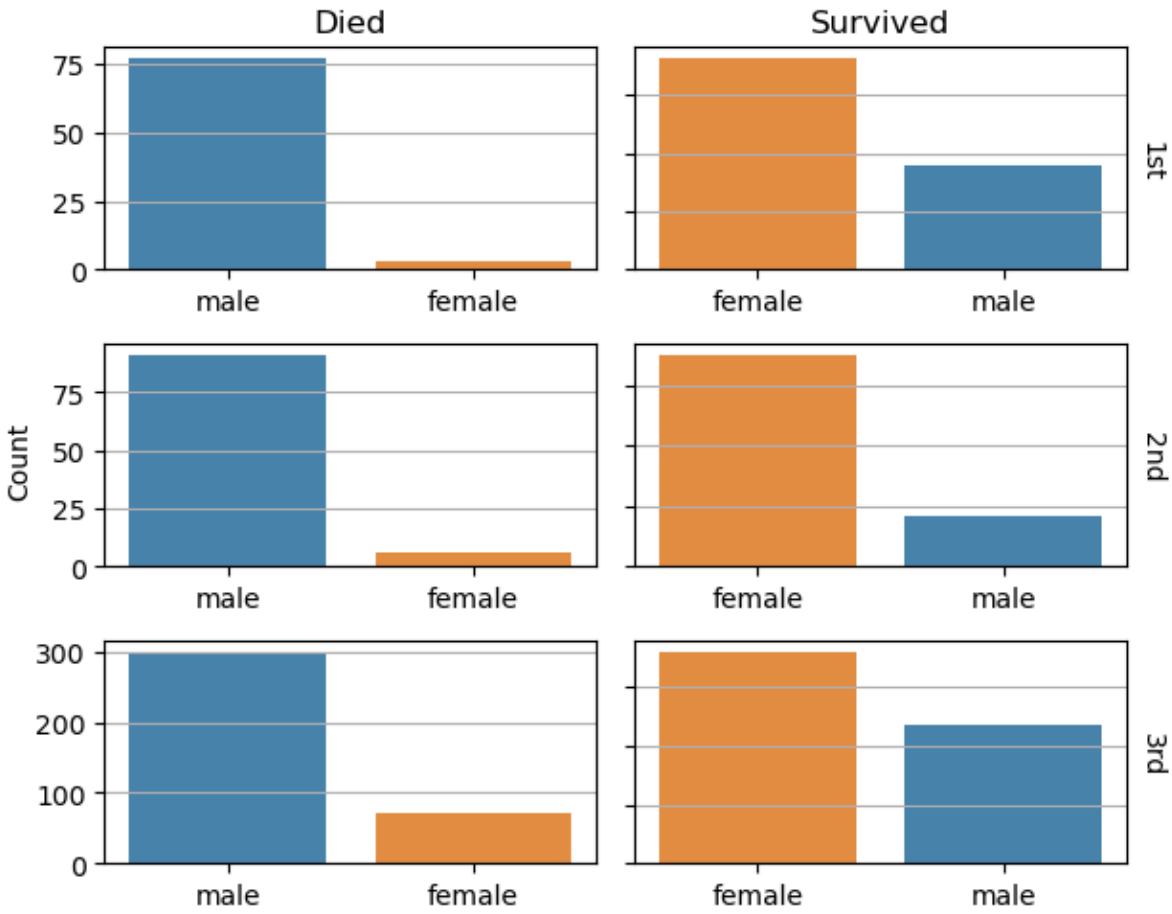
	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cal
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	N
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	N
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C1
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	N
...	...	...	...	...	...	...	...	...	...	...	
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	N
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	E
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	N
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C1
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	N

891 rows × 12 columns

Gráfico de barras

Un gráfico de barras o columnas es una forma de visualizar datos categóricos mediante barras rectangulares de longitud proporcional a los valores representados.

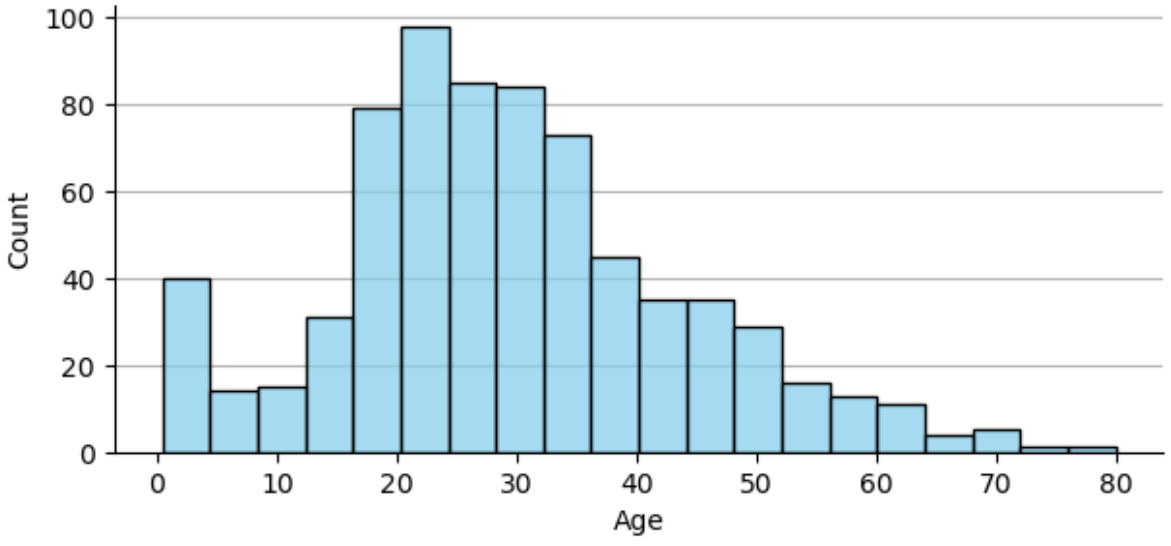
In [4]:
```
titanic_countplot(titanic_df)
```



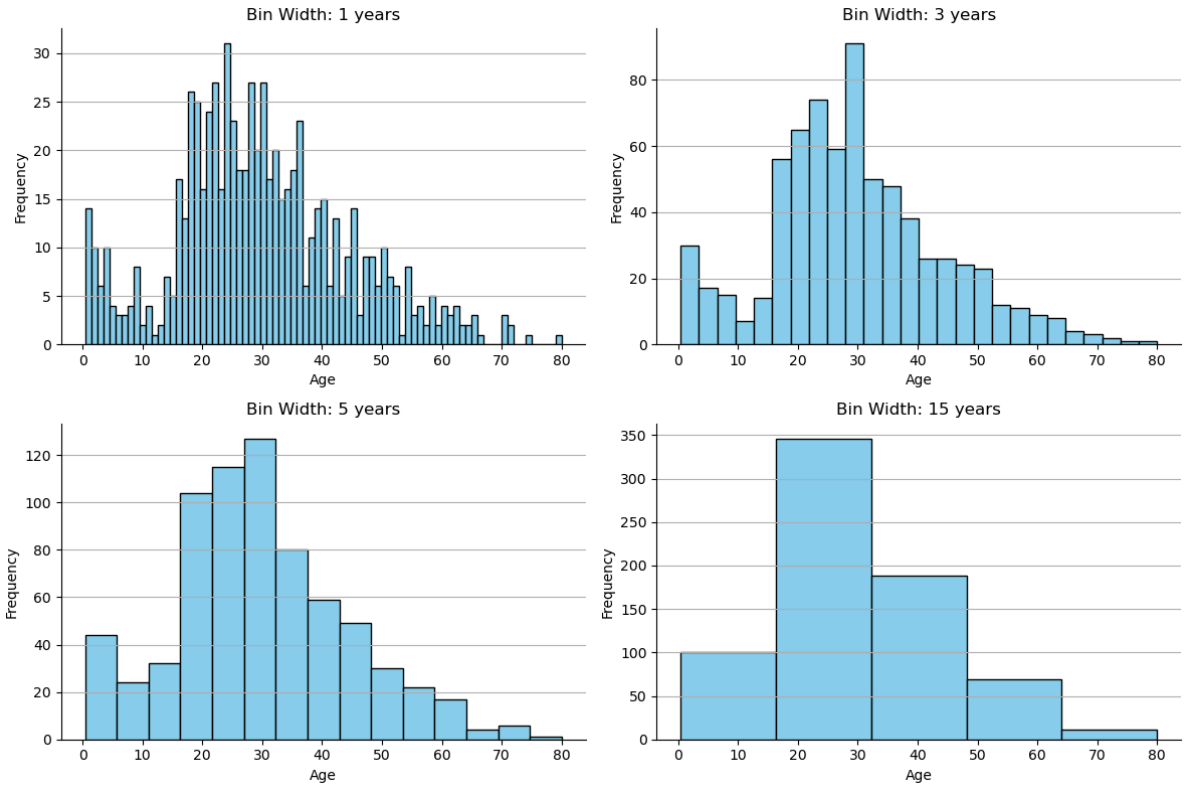
Histogramas

Sirven para visualizar la distribución de datos numéricos, donde el tamaño de cada barra es proporcional a la frecuencia de los valores representados.

In [5]:
```
plt.figure(figsize=(7,3))
sns.histplot(titanic_df['Age'].dropna(), bins=20, kde=False, color='skyblue', edgec
ax = plt.gca();ax.spines['top'].set_visible(False);ax.spines['right'].set_visible(F
plt.grid(axis='y', linestyle='-', which='both');ax.set_axisbelow(True)
plt.show()
```



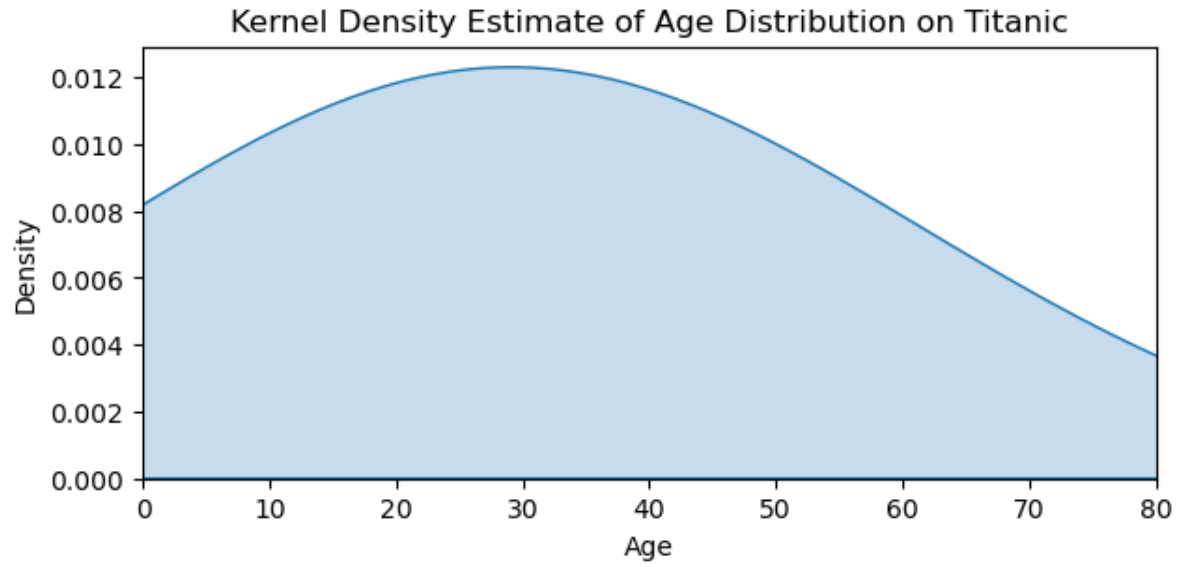
In [6]:
```
titanic_hist(titanic_df)
```



KDE

Un gráfico de estimación de densidad del núcleo (KDE) es un método para visualizar la distribución de observaciones en un conjunto de datos, análogo a un histograma, pero en lugar de usar intervalos discretos, utiliza una función continua (kernel) para suavizar la distribución de los datos.

In [7]:
```
plt.figure(figsize=(7,3))
sns.kdeplot(data=titanic_df['Age'], fill=True, common_norm=False, bw_method=2)
plt.xlabel('Age');plt.ylabel('Density');plt.title('Kernel Density Estimate of Age D
plt.xlim(0,80);plt.show()
```



In [8]:
```
# Load the iris dataset from seaborn
iris = sns.load_dataset('iris')
```

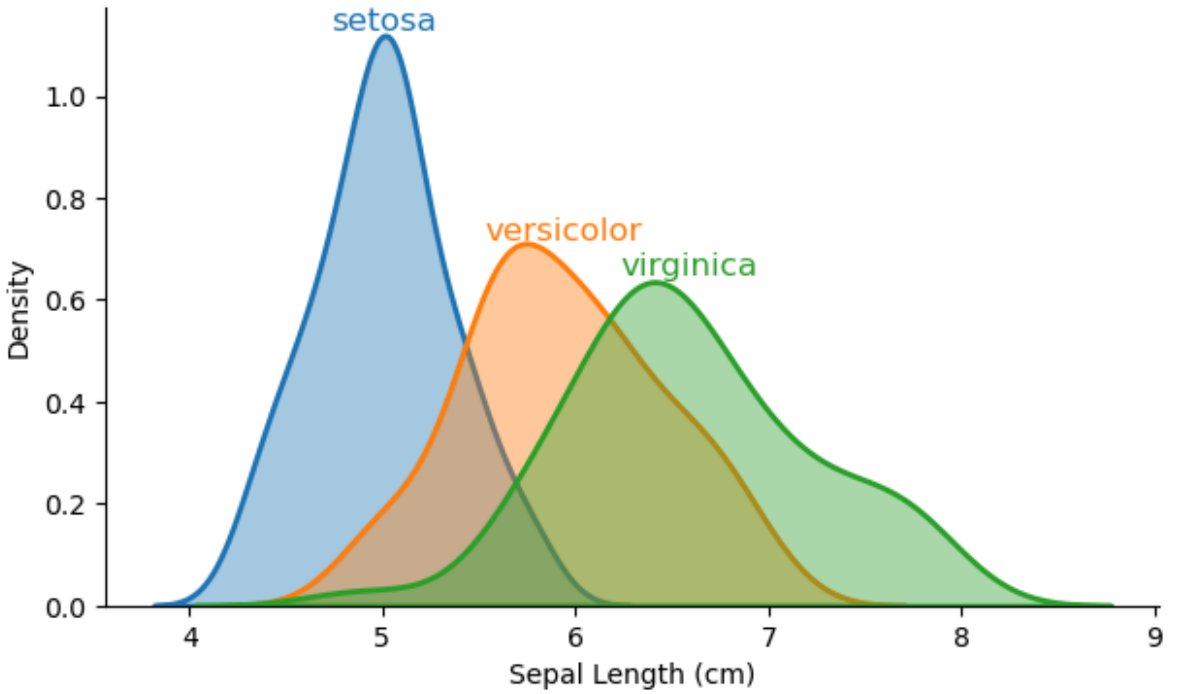
In [9]:
```
iris
```

Out[9]:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

In [10]:
```
iris_color_kde(iris)
```



In [11]:
```
# Load the fuel economy dataset from seaborn
fuel_econ = sns.load_dataset('mpg')
```

In [12]:
```
fuel_econ
```

Out[12]:

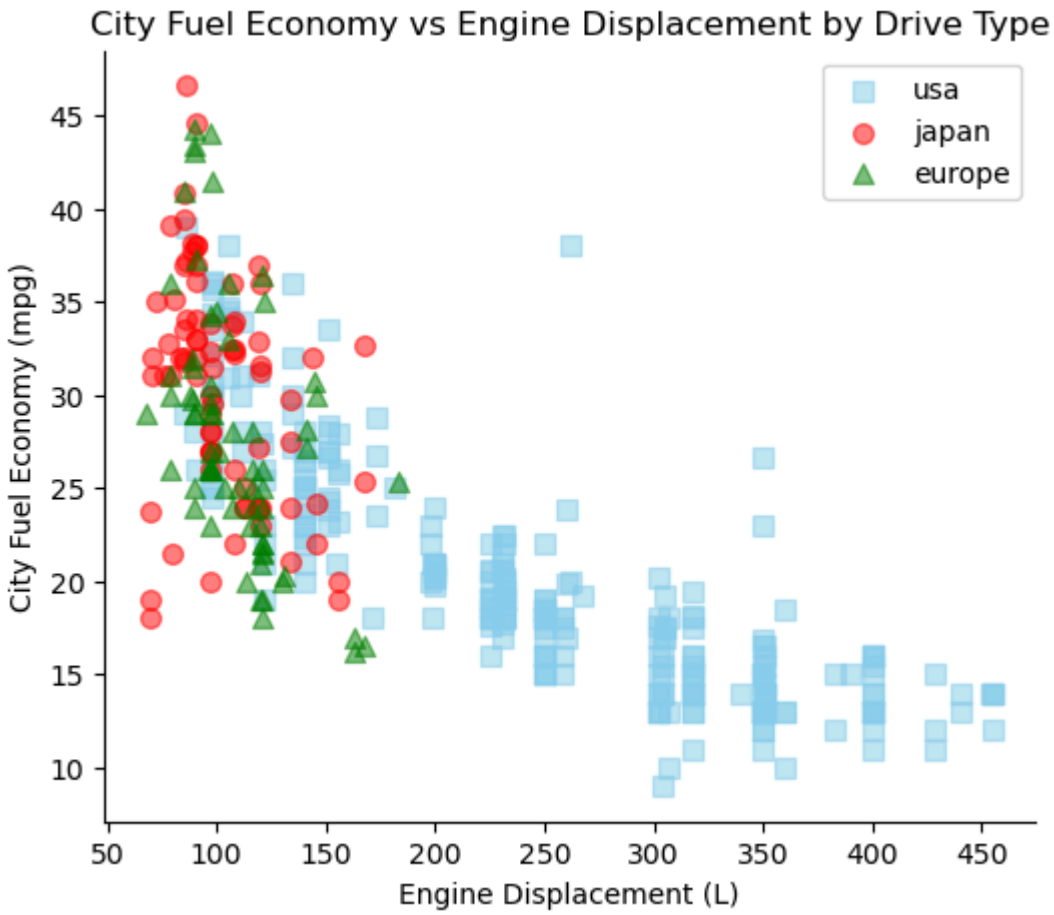
	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	na
0	18.0	8	307.0	130.0	3504	12.0	70	usa	chevrc chev mal
1	15.0	8	350.0	165.0	3693	11.5	70	usa	bu skyl 3
2	18.0	8	318.0	150.0	3436	11.0	70	usa	plymou satel
3	16.0	8	304.0	150.0	3433	12.0	70	usa	a rebel
4	17.0	8	302.0	140.0	3449	10.5	70	usa	f tor
...	...	...	...	...	...	...	...	...	
393	27.0	4	140.0	86.0	2790	15.6	82	usa	f musta
394	44.0	4	97.0	52.0	2130	24.6	82	europe	pick
395	32.0	4	135.0	84.0	2295	11.6	82	usa	doc rampa
396	28.0	4	120.0	79.0	2625	18.6	82	usa	f ran
397	31.0	4	119.0	82.0	2720	19.4	82	usa	chevy

398 rows × 9 columns

Gráfico de dispersión

Se utiliza para mostrar coordenadas con valores continúos en dos dimensiones (x, y).

In [13]:
```
fuel_color_scatter(fuel_econ)
```



In [14]:
```
cities = pd.read_csv('data/california_cities.csv')
```

In [15]:
```
cities
```

Out[15]:

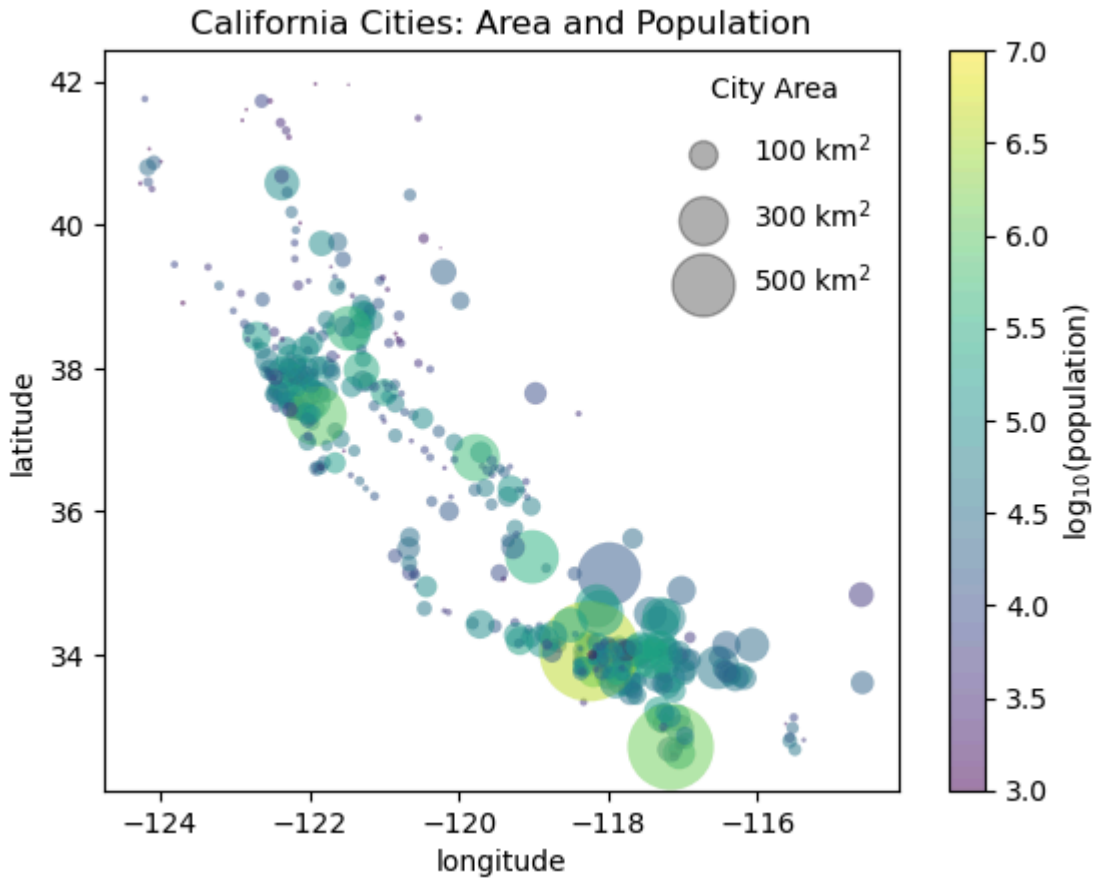
	city	latd	longd	elevation_m	elevation_ft	population_total	area_total_sq_
0	Adelanto	34.576111	-117.432778	875.0	2871.0	31765	56.0
1	AgouraHills	34.153333	-118.761667	281.0	922.0	20330	7.8
2	Alameda	37.756111	-122.274444	NaN	33.0	75467	22.9
3	Albany	37.886944	-122.297778	NaN	43.0	18969	5.4
4	Alhambra	34.081944	-118.135000	150.0	492.0	83089	7.6
...	...	...	...	...	...	...	
477	Yountville	38.403056	-122.362222	30.0	98.0	2933	1.5
478	Yreka	41.726667	-122.637500	787.0	2582.0	7765	10.0
479	YubaCity	39.134722	-121.626111	18.0	59.0	64925	14.6
480	Yucaipa	34.030278	-117.048611	798.0	2618.0	51367	27.8
481	YuccaValley	34.133333	-116.416667	1027.0	3369.0	20700	40.0

482 rows × 13 columns

Burbujas de colores

El tamaño y el color del punto permiten representar otras dos dimensiones.

In [16]:
```
california_scatter(cities)
```



In [17]:
```
# Generate example data
months = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August',
          'September', 'October', 'November', 'December']

# Classify months as 'hot' or 'cold'
hot_months = ['June', 'July', 'August']
cold_months = ['December', 'January', 'February']

# Generate temperatures based on seasons
mean_temperatures = [np.random.uniform(70, 90)
                     if month in hot_months else np.random.uniform(20, 50) for mont

# Generate example standard deviations (replace this with your actual data)
std_dev_temperatures = np.random.uniform(1, 5, len(months))

# Create DataFrame
temperature_df = pd.DataFrame({
    'Month': [month[:3] for month in months],
    'Temperature': mean_temperatures,
    'Std_Dev_Temperature': std_dev_temperatures
})
```

In [18]:
```
temperature_df
```

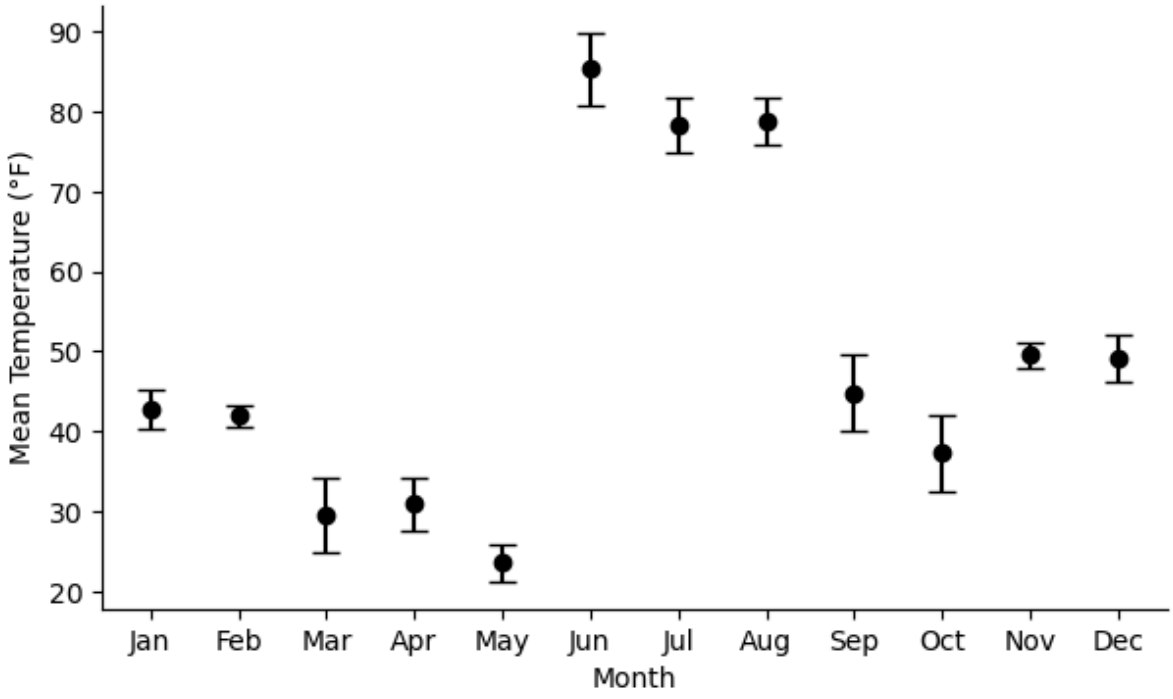
Out[18]:

	Month	Temperature	Std_Dev_Temperature
0	Jan	42.807315	2.524488
1	Feb	41.963764	1.382214
2	Mar	29.639229	4.652820
3	Apr	30.900419	3.266435
4	May	23.579412	2.302511
5	Jun	85.319399	4.559741
6	Jul	78.275784	3.409532
7	Aug	78.771604	2.865663
8	Sep	44.835653	4.706979
9	Oct	37.282010	4.860370
10	Nov	49.589832	1.599467
11	Dec	49.057032	2.913675

Error bar

Las barras de error son representaciones gráficas de la variabilidad de los datos y se utilizan en gráficos para indicar el error o la incertidumbre en una medición informada. Hay diferentes formas de definir las barras de error, y una de ellas es mediante la desviación estándar (cantidad de dispersión con respecto a la media).

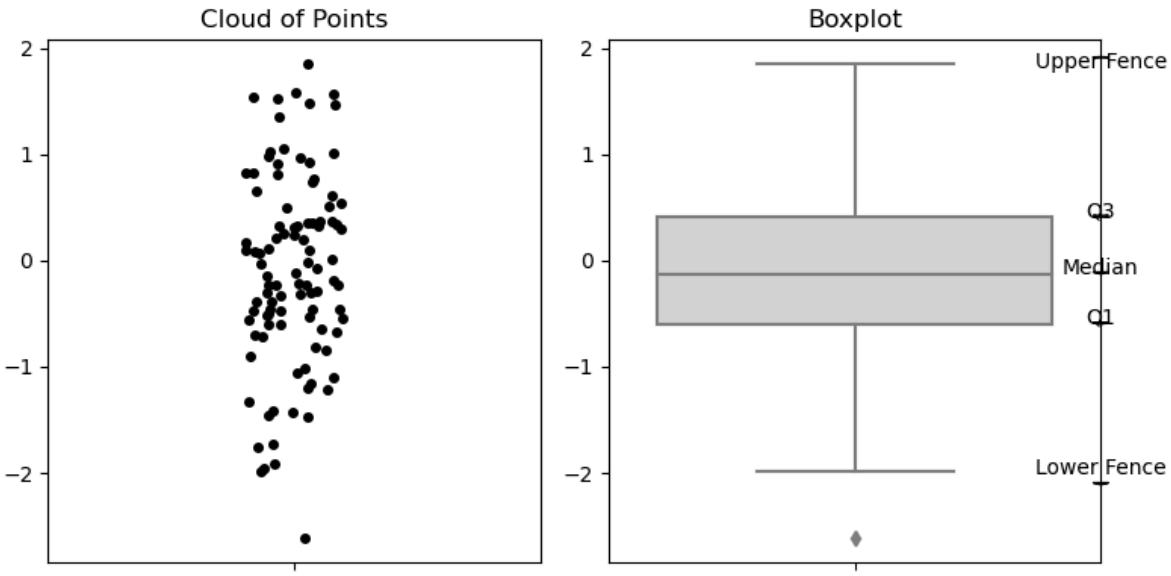
In [19]:
```
temperatures_errorbar(temperature_df)
```



Box plot

Representa gráficamente una serie de datos numéricos a través de la mediana (Q2), los cuartiles superior (Q3) e inferior (Q1), y los valores atípicos (outliers) que están fuera del rango intercuartílico (IQR = Q3 - Q1). Lower = Q1 - 1.5 *IQR y Upper = Q3 + 1.5* IQR.

In [20]:
```
explain_boxplot()
```



In [21]:
```
# Generate example data with more measurements for each month
data = []

# Generate example data
months = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August',
          'September', 'October', 'November', 'December']

# Classify months as 'hot' or 'cold'
hot_months = ['June', 'July', 'August']
cold_months = ['December', 'January', 'February']

# Generate temperatures based on seasons
mean_temperatures = [np.random.uniform(70, 90) if month in hot_months else np.randc

for month in months:
    #temperatures = np.random.uniform(20, 90, 100)  # Generate 100 temperature meas
    temperatures = [np.random.uniform(70, 90) if month in hot_months else np.random
    data.extend(zip([month[:3]] * len(temperatures), temperatures))

temperature_df = pd.DataFrame(data, columns=['Month', 'Temperature'])
```

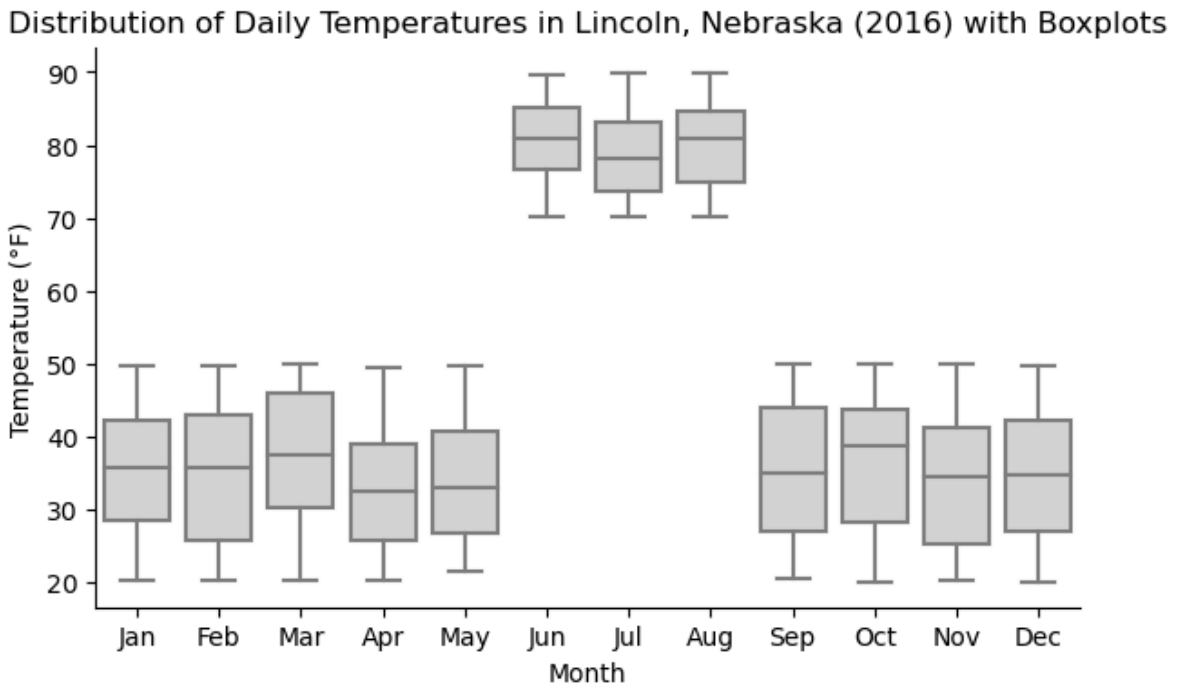
In [22]:
```
temperature_df
```

Out[22]:

	Month	Temperature
0	Jan	47.445932
1	Jan	31.104761
2	Jan	20.463698
3	Jan	47.849557
4	Jan	32.845524
...	...	...
1195	Dec	41.140423
1196	Dec	26.055604
1197	Dec	40.202973
1198	Dec	49.097361
1199	Dec	22.817021

1200 rows × 2 columns

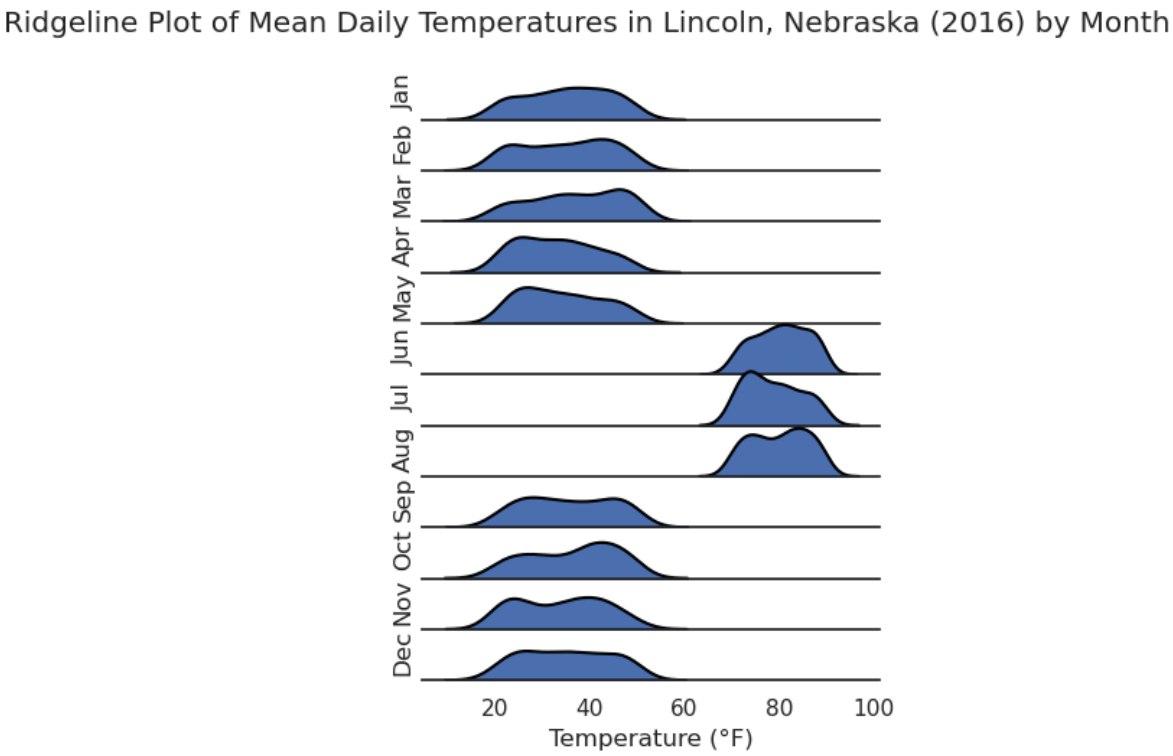
In [23]:
```
temperatures_boxplot(temperature_df)
```



Ridgeline plot

Muestra la distribución de un valor numérico para varios grupos.

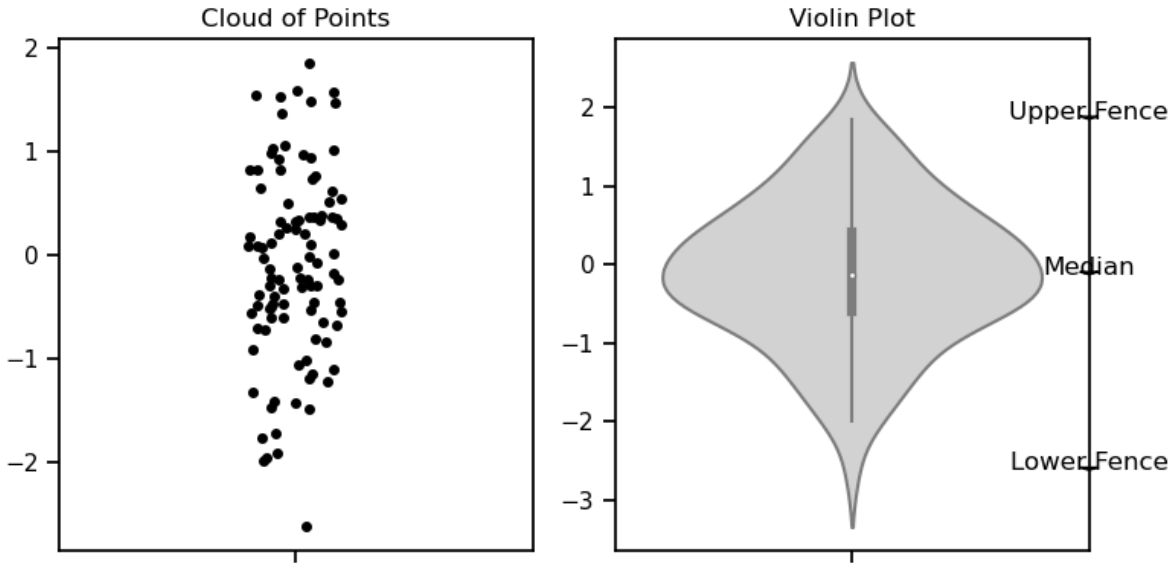
In [24]:
```
temperatures_ridgeline(temperature_df, months)
```



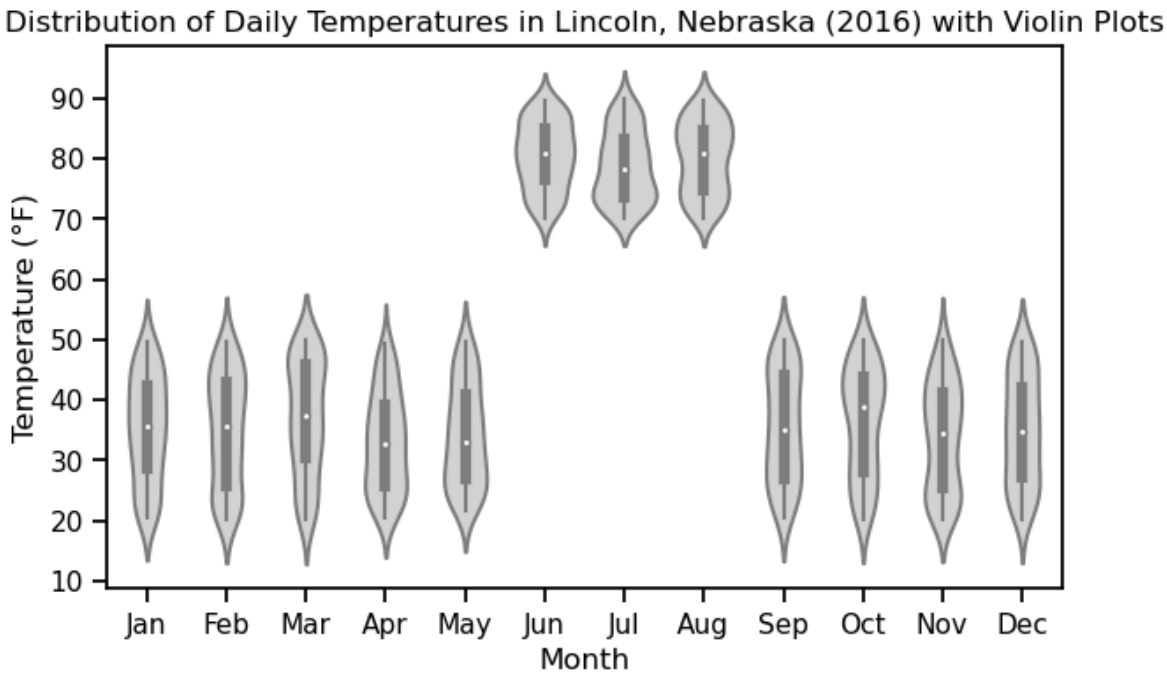
Violin plot

Un diagrama de violín es un gráfico estadístico para comparar distribuciones de probabilidad. Es similar a un box plot, con la adición de un diagrama de densidad kernel rotado en cada lado.

In [25]:
```
explain_violin()
```

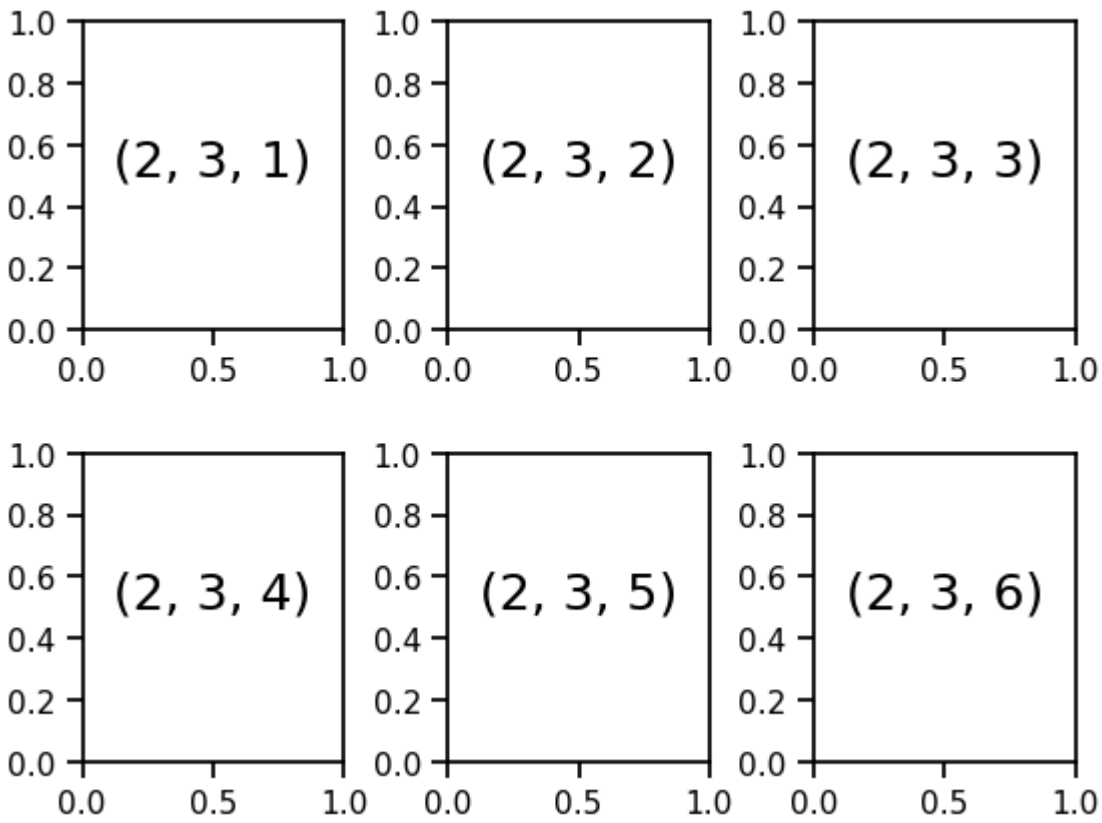


In [26]:
```
temperatures_violin(temperature_df)
```



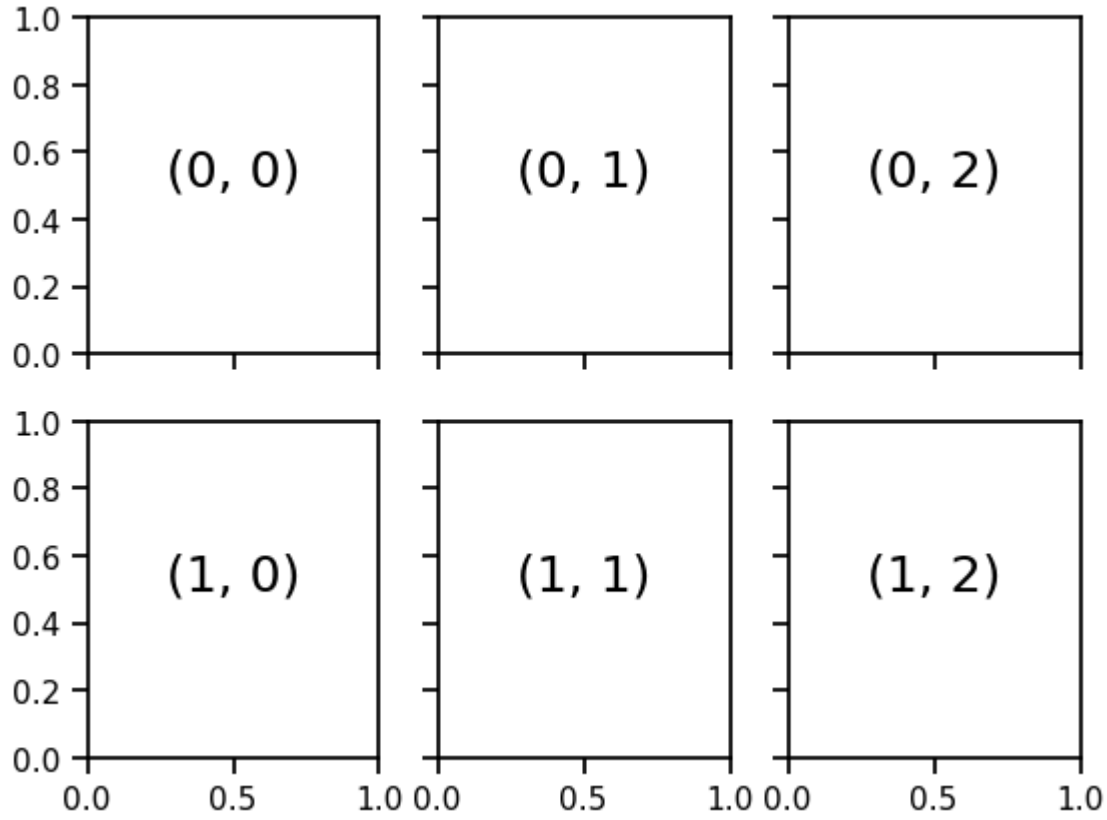
Subplots

In [27]:
```
fig = plt.figure()
fig.subplots_adjust(hspace=0.4, wspace=0.4)
for i in range(1, 7):
    ax = fig.add_subplot(2, 3, i)
    ax.text(0.5, 0.5, str((2, 3, i)), fontsize=18, ha='center')
plt.show()
```



In [28]:
```
fig, ax = plt.subplots(2, 3, sharex='col', sharey='row')
for i in range(2):
    for j in range(3):
```

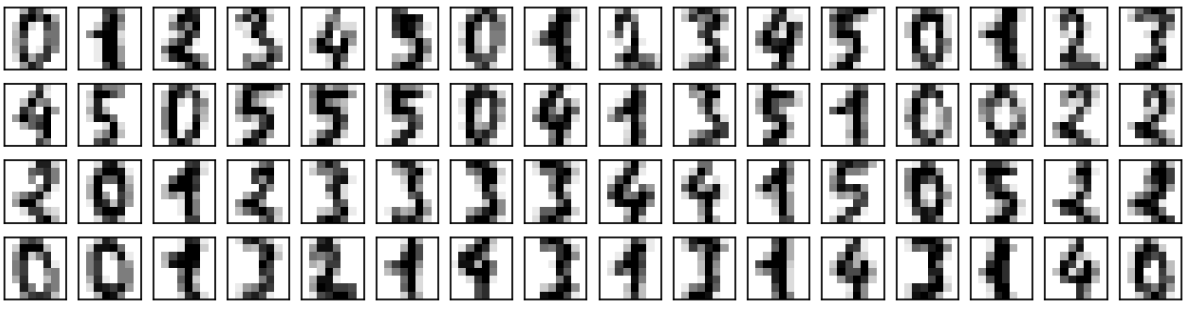
```
        ax[i, j].text(0.5, 0.5, str((i, j)), fontsize=18, ha='center')
plt.show()
```



Imágenes

In [29]:
```
digits = load_digits(n_class=6)

fig, ax = plt.subplots(4, 16, figsize=(16, 4))
for i, axi in enumerate(ax.flat):
    axi.imshow(digits.images[i], cmap='binary')
    axi.set(xticks=[], yticks=[])
plt.show()
```



In [30]:
```
fig, ax = plt.subplots(5, 5, figsize=(5, 5))
fig.subplots_adjust(hspace=0, wspace=0)

faces = fetch_olivetti_faces().images

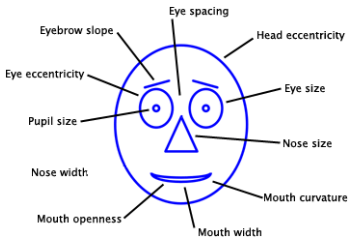
for i in range(5):
    for j in range(5):
        ax[i, j].xaxis.set_major_locator(plt.NullLocator())
        ax[i, j].yaxis.set_major_locator(plt.NullLocator())
        ax[i, j].imshow(faces[10 * i + j], cmap="bone")
```



In [31]:
```
img = io.imread('https://upload.wikimedia.org/wikipedia/commons/thumb/0/00/Crab_Neb
fig = px.imshow(img)
fig.show()
```

Chernoff faces

- Las personas reconocen fácilmente las caras y notan pequeños cambios sin dificultad.
- Los caras de Chernoff modelan datos multivariados en la forma de un rostro humano. Las distintas variables se representan con los ojos, narices, orejas y otras formas de la cara (se dice que es posible incluir hasta 18 dimensiones).



In [32]:
```
dfData=load_usa_arrests_data_frame()
dfData.head(10)
```

Out[32]:

	StateName	Murder	Assault	UrbanPopulation	Rape
0	Alabama	13.2	236	58	21.2
1	Alaska	10.0	263	48	44.5
2	Arizona	8.1	294	80	31.0
3	Arkansas	8.8	190	50	19.5
4	California	9.0	276	91	40.6
5	Colorado	7.9	204	78	38.7
6	Connecticut	3.3	110	77	11.1
7	Delaware	5.9	238	72	15.8
8	Florida	15.4	335	80	31.9
9	Georgia	17.4	211	60	25.8

In [33]:
```
dfData2 = variables_rescale(dfData)
dfData2.head(10)
```

Out[33]:

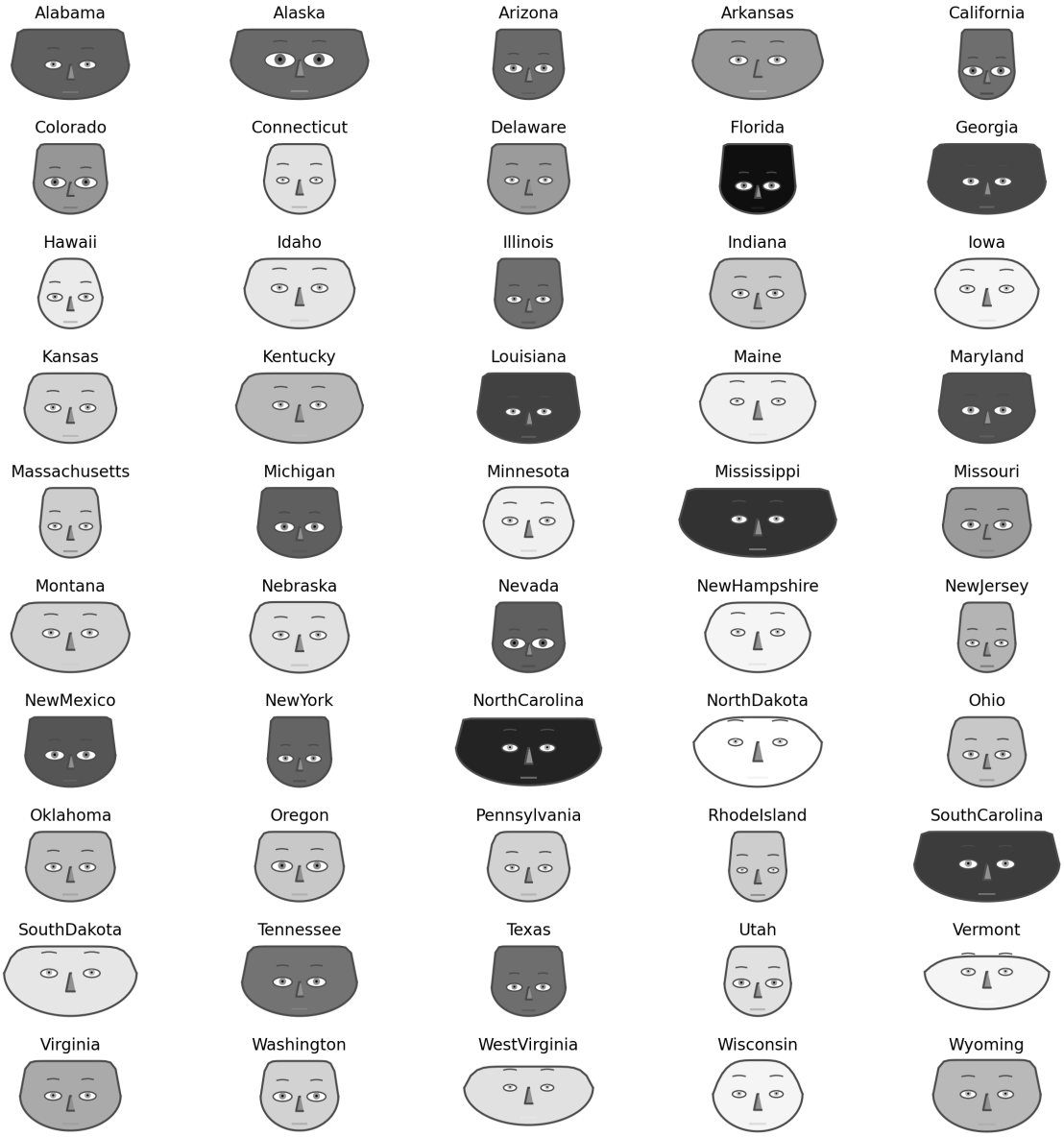
	StateName	Murder	Assault	UrbanPopulation	Rape
0	Alabama	0.746988	0.654110	0.440678	0.359173
1	Alaska	0.554217	0.746575	0.271186	0.961240
2	Arizona	0.439759	0.852740	0.813559	0.612403
3	Arkansas	0.481928	0.496575	0.305085	0.315245
4	California	0.493976	0.791096	1.000000	0.860465
5	Colorado	0.427711	0.544521	0.779661	0.811370
6	Connecticut	0.150602	0.222603	0.762712	0.098191
7	Delaware	0.307229	0.660959	0.677966	0.219638
8	Florida	0.879518	0.993151	0.813559	0.635659
9	Georgia	1.000000	0.568493	0.474576	0.478036

In [34]:
```
dfData2.sort_values(by="Murder").tail(10)
```

Out[34]:

	StateName	Murder	Assault	UrbanPopulation	Rape
27	Nevada	0.686747	0.708904	0.830508	1.000000
42	Texas	0.716867	0.534247	0.813559	0.470284
32	NorthCarolina	0.734940	1.000000	0.220339	0.227390
41	Tennessee	0.746988	0.489726	0.457627	0.506460
0	Alabama	0.746988	0.654110	0.440678	0.359173
39	SouthCarolina	0.819277	0.801370	0.271186	0.392765
8	Florida	0.879518	0.993151	0.813559	0.635659
17	Louisiana	0.879518	0.698630	0.576271	0.385013
23	Mississippi	0.921687	0.732877	0.203390	0.253230
9	Georgia	1.000000	0.568493	0.474576	0.478036

In [35]:
```
fig = chernoff_face(data=dfData2,
                    n_columns=5,
                    long_face=False,
                    color_mapper=matplotlib.cm.Greys,
                    figsize=(12, 12), dpi=200)
fig.tight_layout()
matplotlib.pyplot.show()
```



Treemap

Permite visualizar datos jerárquicos utilizando rectángulos anidados.

In [36]:
```
df = px.data.gapminder().query("year == 2007")
fig = px.treemap(df, path=[px.Constant("world"), 'continent', 'country'], values='p
                 color='lifeExp', hover_data=['iso_alpha'], color_continuous_scale
                 color_continuous_midpoint=np.average(df['lifeExp'], weights=df['p
fig.update_layout(margin = dict(t=50, l=25, r=25, b=25))
fig.show()
```



In [37]:
```
df = pd.read_csv("data/datos.csv")
columns_of_interest = ["Edad", "Altura", "Peso"]
subset_df = df[columns_of_interest]
```

In [38]:
```
subset_df.head()
```

Out[38]:

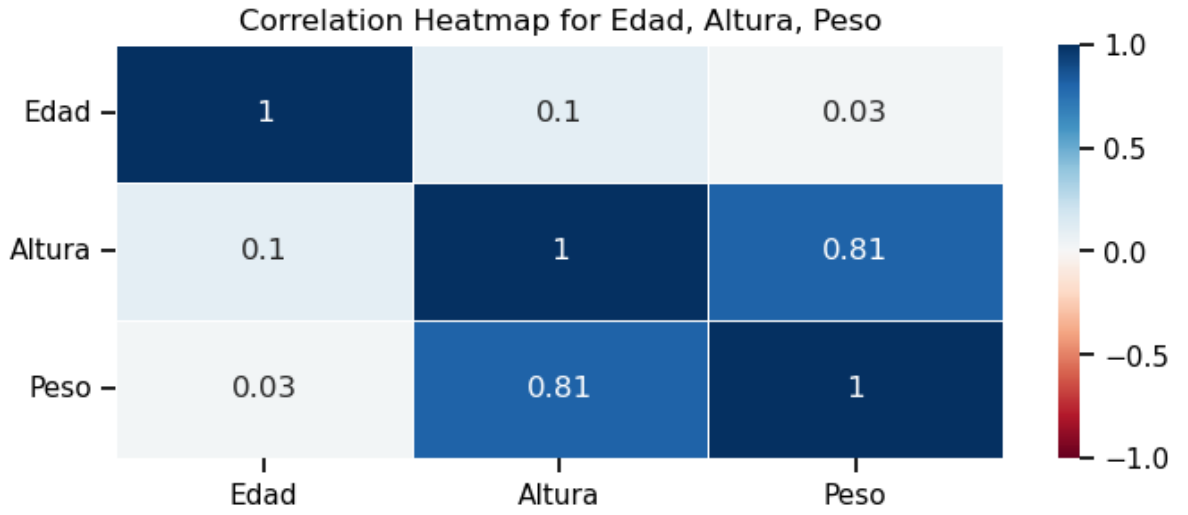
	Edad	Altura	Peso
0	85	176.47	81
1	57	149.48	59
2	58	185.76	85
3	81	156.99	60
4	70	178.13	75

Análisis de correlación

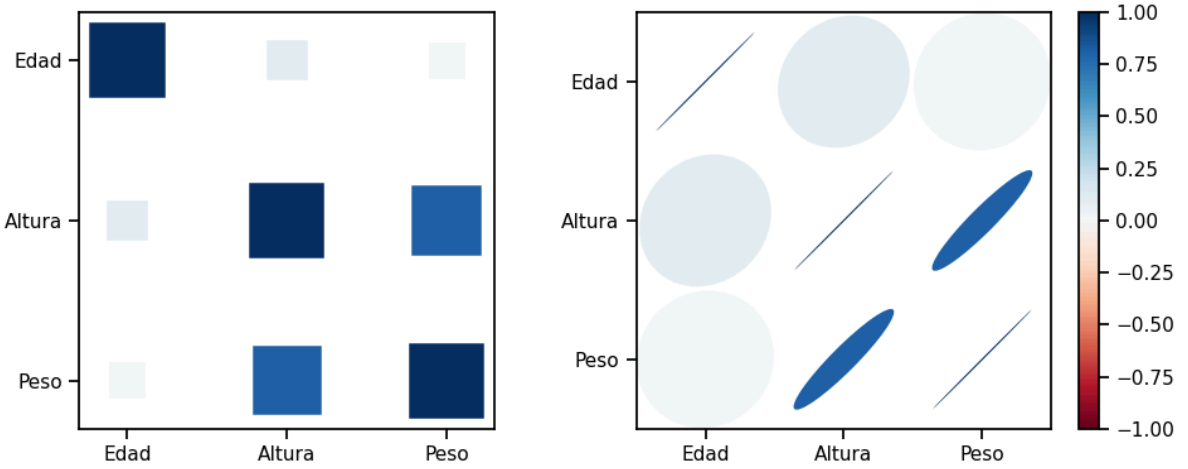
La matriz de correlación es una herramienta muy útil para analizar la relación entre características.

In [39]:
```
plt.figure(figsize=(8, 3))
sns.heatmap(subset_df.corr(), annot=True, cmap='RdBu', square=False, linewidths=.5,
plt.yticks(rotation=0)
```

```
plt.title("Correlation Heatmap for {}".format(", ".join(columns_of_interest).strip(
plt.show()
```



In [40]:
```
plot_corr(subset_df.corr(), figsize=(10, 4), marker='s', cmap='RdBu')
```



Nube de palabras

Representación visual de las palabras que conforman un texto de modo que las más frecuentes se muestran con mayor prominencia.

In [41]:
```
wordcloud = WordCloud(background_color='white', width=1500, height=1000).generate('
plt.figure(figsize=(5,5))
plt.imshow(wordcloud)
plt.axis('off')
plt.show()
```



In [42]:
```
dictionary = {'a':5, 'b':3, 'c':2, 'd':1, 'e':1}
wordcloud = WordCloud(background_color='black',
                      width=1500,
                      height=1000
                      ).generate_from_frequencies(dictionary)

plt.figure(figsize=(5,5))
plt.imshow(wordcloud)
plt.axis('off')
plt.show()
```



Nube de palabras de un libro

In [43]:
```
with open('data/holmes.txt') as f:
    text = f.read().lower()
text[:2000]
```

Out[43]:
```
'\n\n\n\n                          the complete sherlock holmes\n\n
arthur conan doyle\n\n\n\n                                table of contents\n\n
a study in scarlet\n\n              the sign of the four\n\n                  the
adventures of sherlock holmes\n              a scandal in bohemia\n
the red-headed league\n              a case of identity\n              the bosco
mbe valley mystery\n              the five orange pips\n              the man wi
th the twisted lip\n              the adventure of the blue carbuncle\n
the adventure of the speckled band\n              the adventure of the engineer
\'s thumb\n              the adventure of the noble bachelor\n              the
adventure of the beryl coronet\n              the adventure of the copper beeches
\n\n                  the memoirs of sherlock holmes\n              silver blaze
\n              the yellow face\n              the stock-broker\'s clerk\n
the "gloria scott"\n              the musgrave ritual\n              the reigate
squires\n              the crooked man\n              the resident patient\n
the greek interpreter\n              the naval treaty\n              the final p
roblem\n\n                  the return of sherlock holmes\n              the adve
nture of the empty house\n              the adventure of the norwood builder\n
the adventure of the dancing men\n              the adventure of the solitary cyc
list\n              the adventure of the priory school\n              the advent
ure of black peter\n              the adventure of charles augustus milverton\n
the adventure of the six napoleons\n              the adventure of the three stud
ents\n              the adventure of the golden pince-nez\n              the adv
enture of the missing three-quarter\n              the adventure of the abbey gra
nge\n              the adventure of the second stain\n\n              the hound
of the baskervi'
```

Máscara

In [44]:
```
mask = np.array(Image.open('img/holmes.png'))
```

In [45]:
```
plt.imshow(mask)
plt.axis('off')
plt.show()
```



In [46]:
```
wordcloud = WordCloud(mask=mask,background_color="black").generate(text)
plt.figure(figsize=(9,6))
plt.imshow(wordcloud)
plt.axis('off')
plt.show()
```



Mapas

In [47]:
```
df_glob = px.data.gapminder()
```

In [48]:
```
df_glob
```

Out[48]:

	country	continent	year	lifeExp	pop	gdpPercap	iso_alpha	iso_num
0	Afghanistan	Asia	1952	28.801	8425333	779.445314	AFG	4
1	Afghanistan	Asia	1957	30.332	9240934	820.853030	AFG	4
2	Afghanistan	Asia	1962	31.997	10267083	853.100710	AFG	4
3	Afghanistan	Asia	1967	34.020	11537966	836.197138	AFG	4
4	Afghanistan	Asia	1972	36.088	13079460	739.981106	AFG	4
...	...	...	...	...	...	...	...	...
1699	Zimbabwe	Africa	1987	62.351	9216418	706.157306	ZWE	716
1700	Zimbabwe	Africa	1992	60.377	10704340	693.420786	ZWE	716
1701	Zimbabwe	Africa	1997	46.809	11404948	792.449960	ZWE	716
1702	Zimbabwe	Africa	2002	39.989	11926563	672.038623	ZWE	716
1703	Zimbabwe	Africa	2007	43.487	12311143	469.709298	ZWE	716

1704 rows × 8 columns

In [49]:
```
fig = px.scatter_geo(df_glob, locations="iso_alpha", color="continent", hover_name=
                     animation_frame="year", projection="natural earth")
fig.show()
```

In [50]:
```
fig = px.choropleth(df_glob, locations="iso_alpha", color="lifeExp", hover_name="cc
                    animation_frame="year", range_color=[20,80])
```

```
fig.show()
```



In [51]:
```
df_year = df_glob.query("year == 2007")
```

In [52]:
```
df_year.head()
```

Out[52]:

	country	continent	year	lifeExp	pop	gdpPercap	iso_alpha	iso_num
11	Afghanistan	Asia	2007	43.828	31889923	974.580338	AFG	4
23	Albania	Europe	2007	76.423	3600523	5937.029526	ALB	8
35	Algeria	Africa	2007	72.301	33333216	6223.367465	DZA	12
47	Angola	Africa	2007	42.731	12420476	4797.231267	AGO	24
59	Argentina	Americas	2007	75.320	40301927	12779.379640	ARG	32

In [53]:
```
fig = px.scatter_geo(df_year, locations="iso_alpha", color="continent",
    size="pop", hover_name="country", projection="orthographic")
fig.show()
```


Pan